UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

WORKSHEET 6

Student Name: Aradhya Sharma **UID: 25MCC20042**

Branch: MCA (CCD) **Section/Group: 25MCC-101A**

Semester: 2 **Date of Performance: 24 FEB 2026**

Subject Name: Technical training **Subject Code: 25CAP-652**

1. AIM

To learn how to **create, query, and manage views** in PostgreSQL in order to simplify database queries and provide a secure abstraction layer for end users.

---

2. OBJECTIVES

- To understand data abstraction using SQL views
- To hide complex queries behind virtual tables
- To simplify multi-table queries for reporting
- To implement views for restricted data access
- To strengthen SQL skills used in enterprise systems

---

3. SOFTWARE REQUIREMENTS

Operating System:
- Windows 10 / 11 / Linux / macOS

Database Management System (DBMS):
- PostgreSQL (version 12 or higher)

SQL Interface:
- pgAdmin 4

OR
- PostgreSQL Command Line Interface (psql)

4. PROCEDURE FOR EXPERIMENT

Step 1: Creating a Simple View for Data Filtering

Objective:
To create a simple view that displays only active employees, hiding the complete employee table structure.

```
CREATE TABLE departments (
   dept_id INT PRIMARY KEY,
   dept_name TEXT
);

CREATE TABLE employees (
   emp_id INT PRIMARY KEY,
   emp_name TEXT,
   salary NUMERIC,
   status TEXT,
   dept_id INT REFERENCES departments(dept_id)
);

CREATE VIEW active_employees_view AS
SELECT emp_id, emp_name, dept_id
FROM employees
WHERE status = 'Active';
SELECT * FROM active_employees_view;
```

Output:

	emp_id integer	emp_name text	dept_id integer
1	101	Amit	1
2	103	Rahul	2
3	104	Neha	1
4	105	Arjun	3

Step 2: Creating a View for Joining Multiple Tables

Objective:
To simplify retrieval of employee and department information using a join-based view.

```
CREATE VIEW employee_department_view AS
SELECT
   e.emp_id,
   e.emp_name,
   d.dept_name,
   e.salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id;

SELECT * FROM employee_department_view;
```

Output:

	emp_id integer	emp_name text	dept_name text	salary numeric
1	101	Amit	HR	42000
2	102	Priya	Engineering	36000
3	103	Rahul	Engineering	55000
4	104	Neha	HR	30000
5	105	Arjun	Finance	48000
6	106	Rohit	Finance	39000

Step 3: Advanced Summarization View

Objective:
To create an aggregated view that provides department-level statistics automatically.

```
CREATE VIEW department_summary_view AS
SELECT
   d.dept_name,
```

```
    COUNT(e.emp_id) AS total_employees,
    AVG(e.salary)::NUMERIC(20, 2) AS avg_salary,
    SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_name;

SELECT * FROM department_summary_view;
```

Output:

	dept_name text	total_employees bigint	avg_salary numeric (20,2)	total_salary numeric
1	Finance	2	43500.00	87000
2	Engineering	2	45500.00	91000
3	HR	2	36000.00	72000

Learning Outcome

This experiment demonstrates the effective use of **views in PostgreSQL** to provide abstraction, simplify complex queries, and enhance data security.
Students gain practical exposure to view-based design patterns commonly used in:

- Payroll systems
- Reporting and analytics platforms
- Enterprise database security
- Placement-oriented technical interviews